

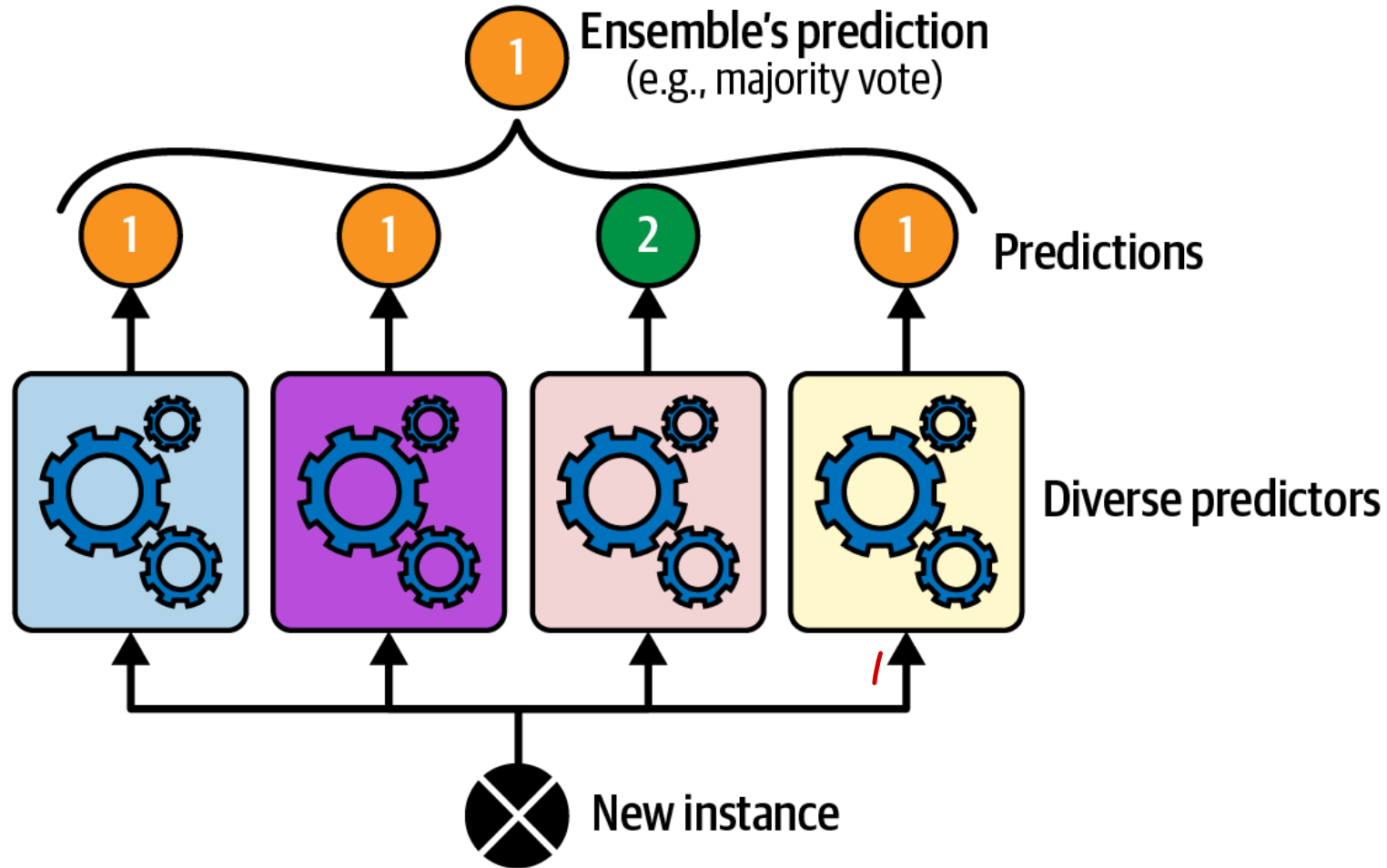
Topic 7

Random Forest

Ensemble Models

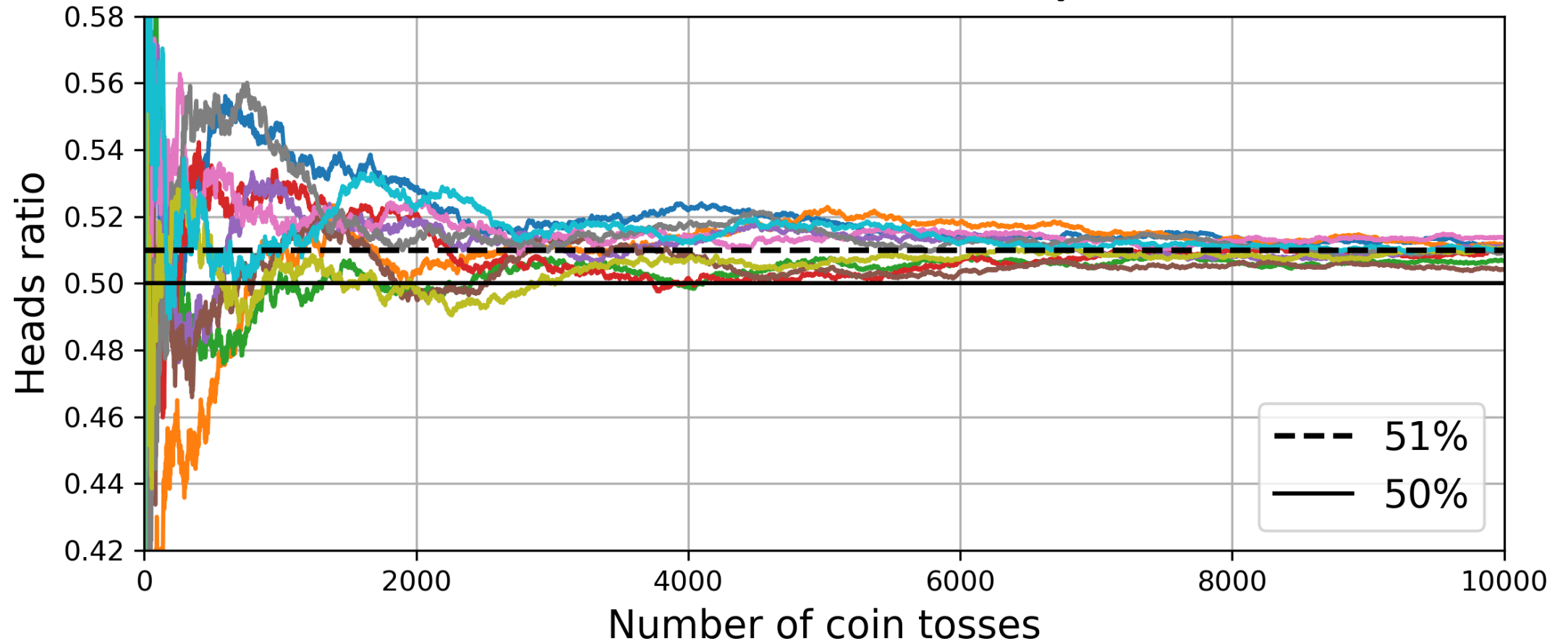
- An aggregate prediction
- Wisdom of a crowd !
- Train a group of decision trees each on a random subset of a training set.
- Ensemble of decision trees is called a random forest.

Voting Classifiers



- Hard Voting Classifier – a majority voting classifier

Biased Heads Example



- Coin with a heads 51% of the time ... the probability of getting a heads majority of the time increases as the number of coin tosses increases.

Ensemble Models

ML

- Just as tossing coins are independent events, ensemble models work the best when the predictors are as independent from each other as possible.
- One way to get diverse classifiers is to train them using very different algorithms. This increases the chance that they will make very different types of errors, improving the ensemble's accuracy.

Voting Classifiers Code

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

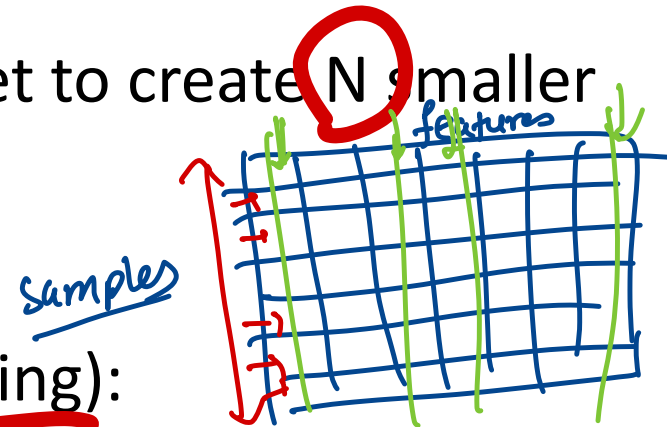
X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)
```

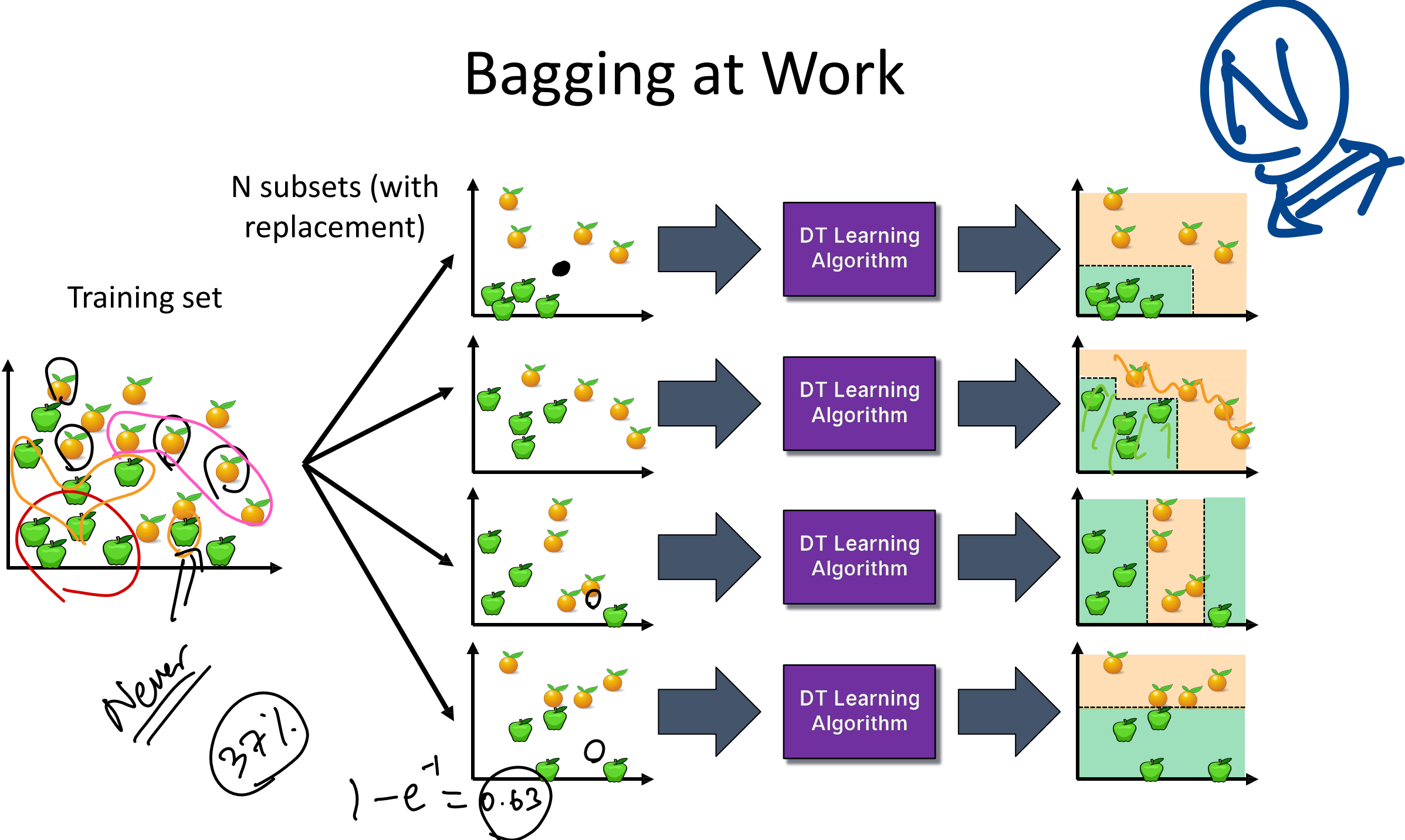
- Soft Voting is a probability based voting classifier, giving higher weight to highly confident votes.

Random Forests

- Random Forests:
 - Instead of building a single decision tree and use it to make predictions, build many slightly different trees and combine their predictions
- We have a single data set, so how do we obtain slightly different trees?
 1. Bagging (**Bootstrap Aggregating**):
 - Take random subsets of data points from the training set to create **N** smaller data sets
 - Fit a decision tree on each subset
 2. Random Subspace Method (also known as **Feature Bagging**):
 - Fit **N** different decision trees by constraining each one to operate on a random subset of features



Bagging at Work



Bagging at Work - Training

How to build and evaluate a random forest?

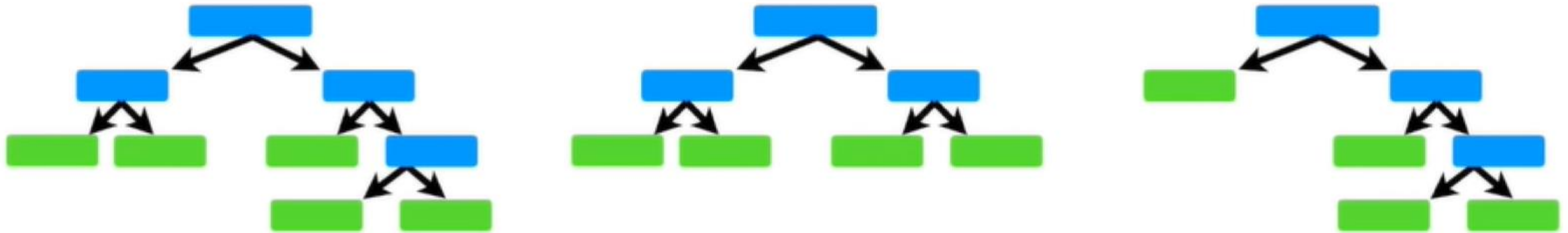
1. Create a Bootstrapped data set from the original data set

2. Build Trees → Tree Creation

3. Run Data Along Each Tree → Train

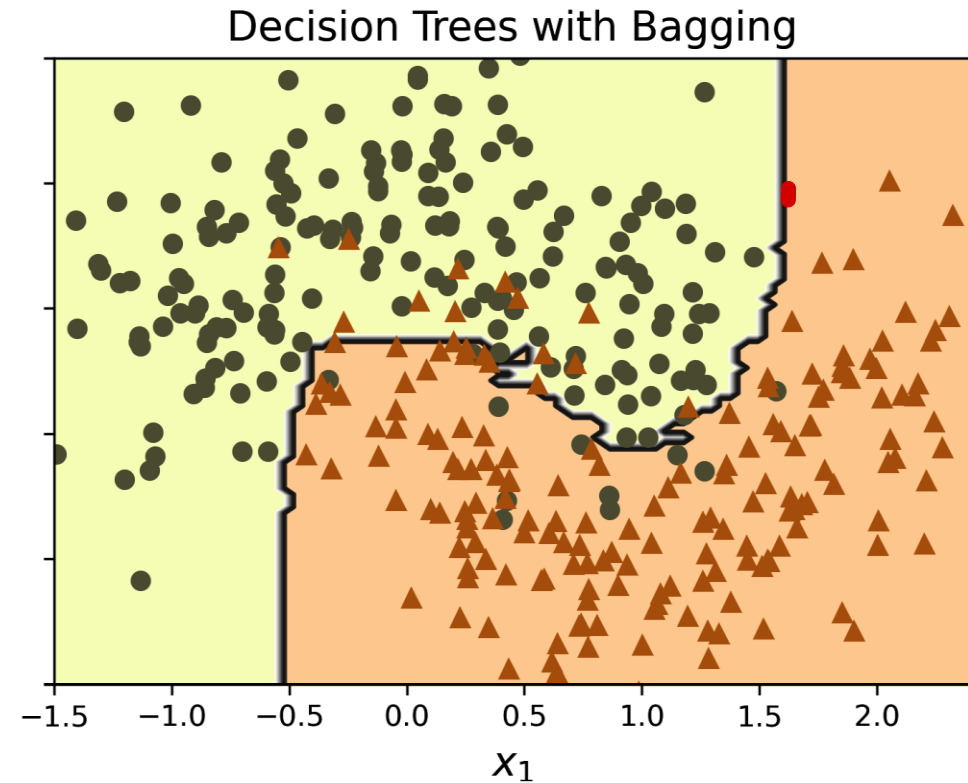
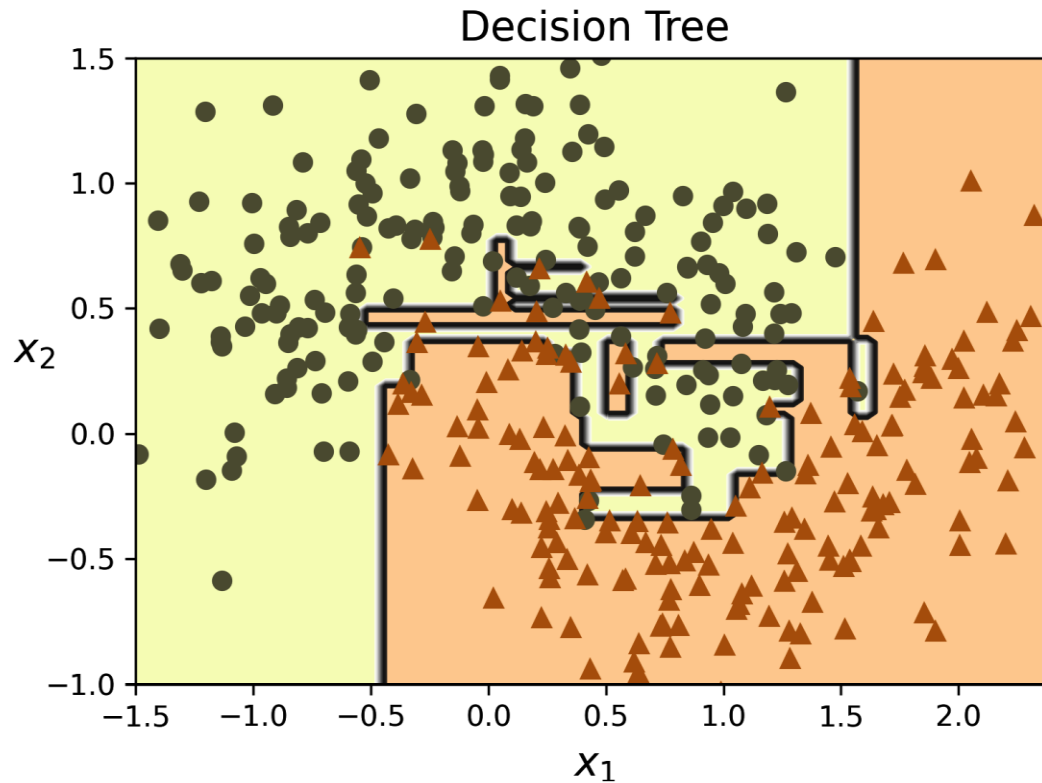
Random Selection

The good news is that **Random Forests** combine the simplicity of decision trees with flexibility resulting in a vast improvement in accuracy.



```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
```

```
bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,  
                             max_samples=100, n_jobs=-1, random_state=42)  
bag_clf.fit(X_train, y_train)
```



Bagging at Work - Training

Bootstrap

Step 1.

Create a Bootstrapped Database

Original Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Samples

Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
------------	------------------	------------------	--------	---------------

To create a bootstrapped dataset that is the same size as the original, we just randomly select samples from the original dataset.

The important detail is that we're allowed to pick the same sample more than one with replacement

Bagging at Work - Training

Bootstrap

Step 2: Create a decision tree using the bootstrapped dataset, but only use a random subset of variables (or columns) at each step.

In this example, we will only consider 2 variables (columns) at each step.

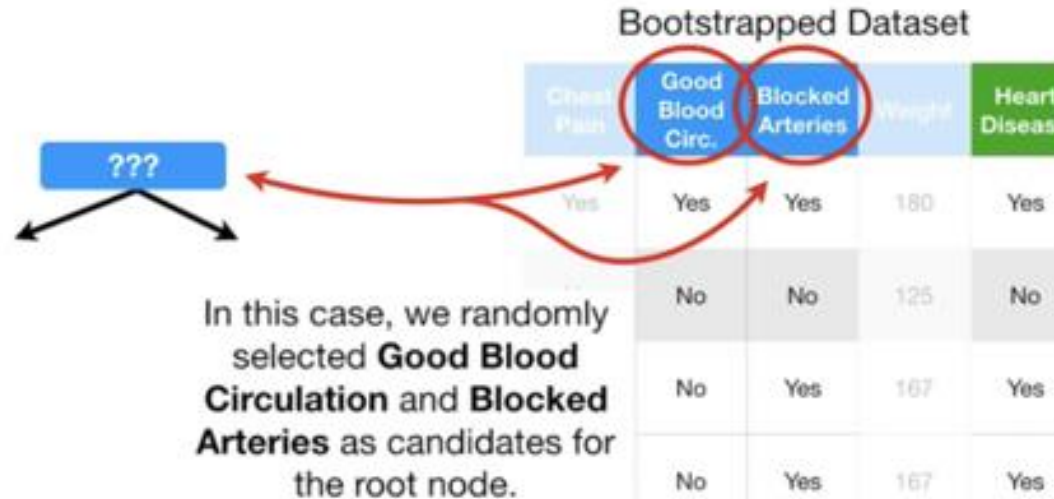
NOTE: We'll talk more about how to determine the optimal number of variables to consider later...

Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

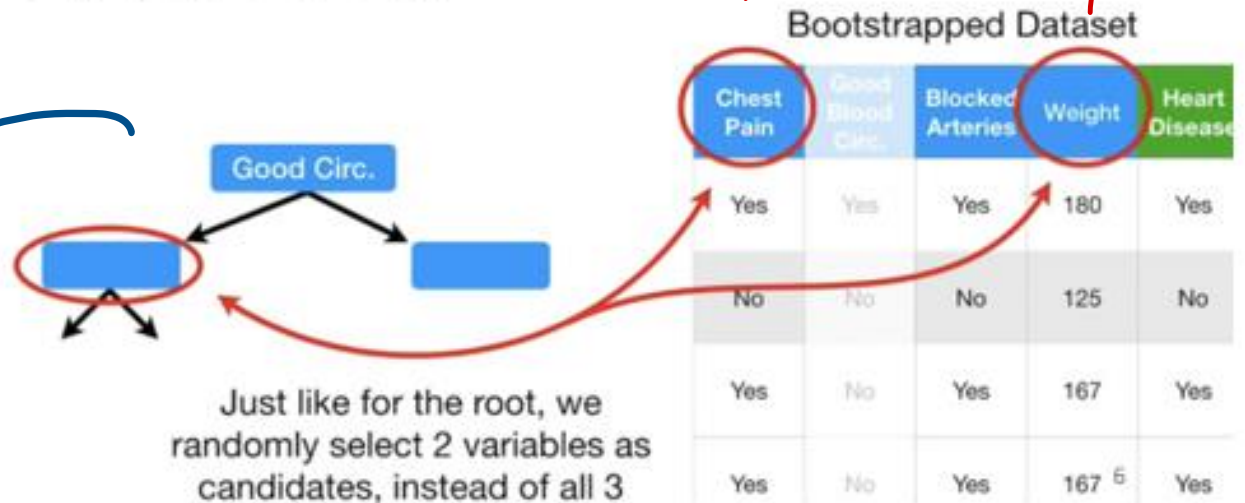
Bagging at Work - Training

Bootstrap



Randomly Selected
Samples
To be passed into DTs.

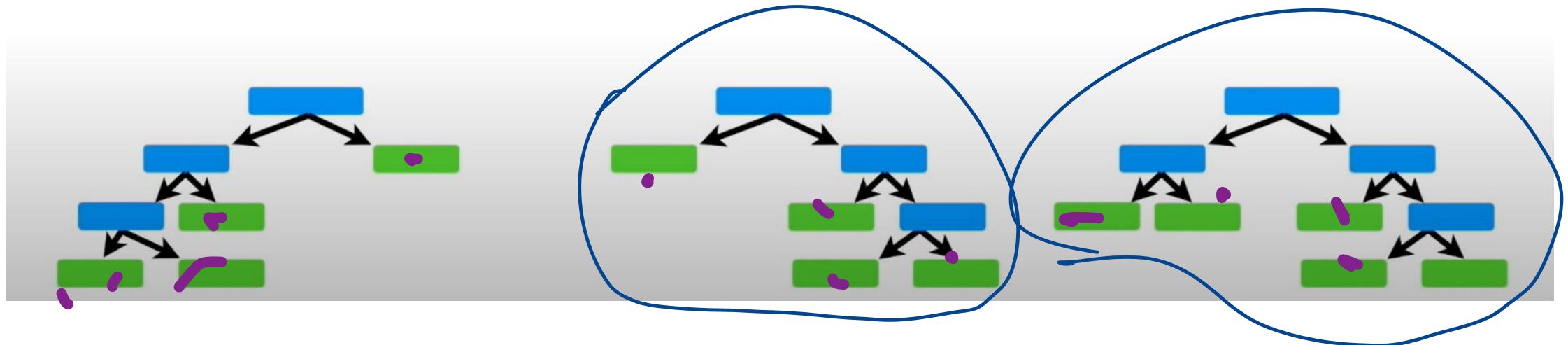
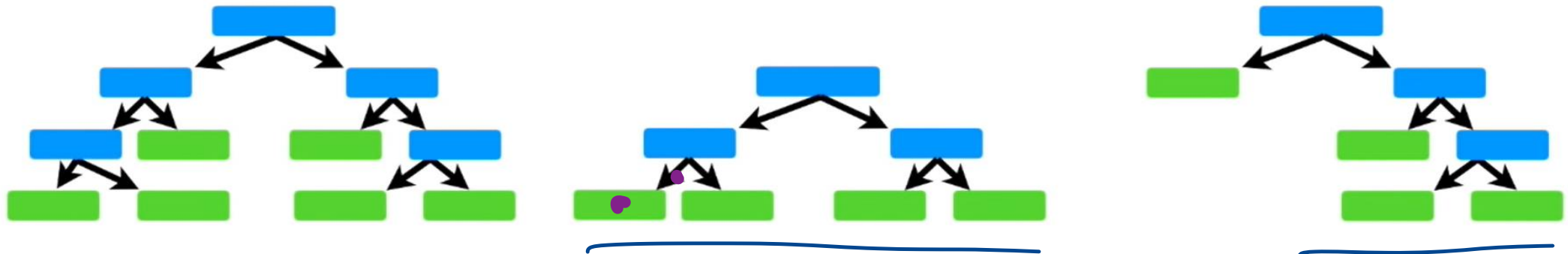
Root Node of DT



Bagging at Work - Training

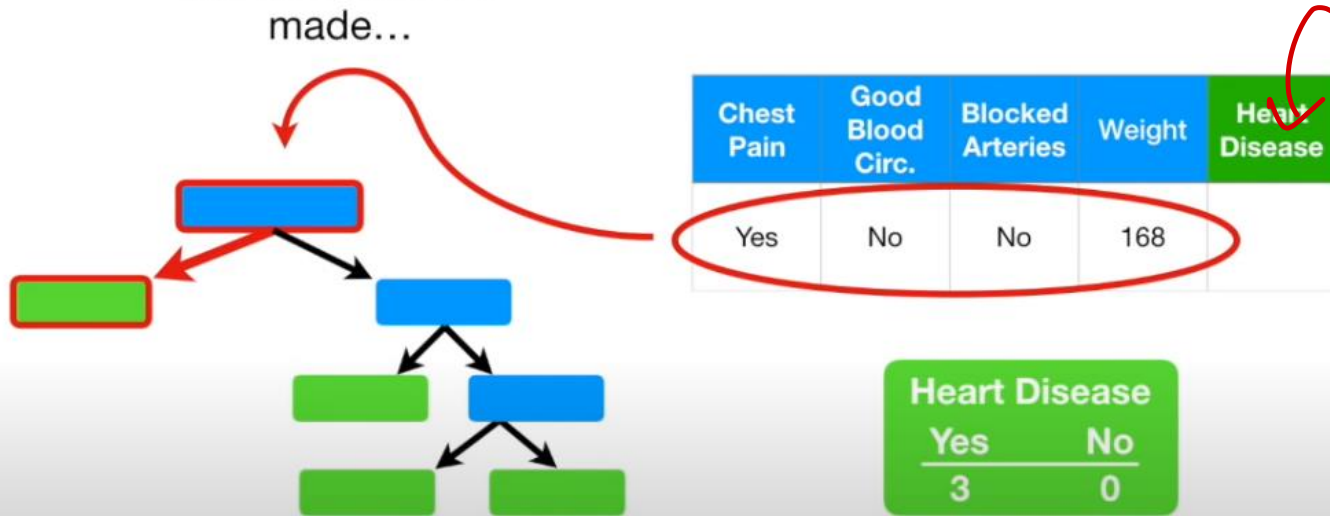
Bootstrap

Now go back to Step 1 and repeat: Make a new bootstrapped dataset and build a tree considering a subset of variables at each step.



Bagging at Work - Testing

Then we repeat for all the trees that we made...



Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	No	168	

...and now we want to know if they have heart disease or not.

code
218/219

Out of Bag Evaluation

100

m

Original Dataset

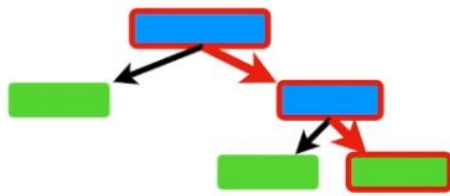
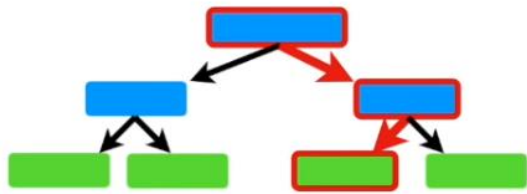
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Here is the entry that didn't end up in the bootstrapped dataset..

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	No	210	No

- 63% of the training instances are sampled on average for each predictor.
- 37% of training instances are not sampled are called out of bag (OOB) instances.

- Run the out of bag sample through the trees.

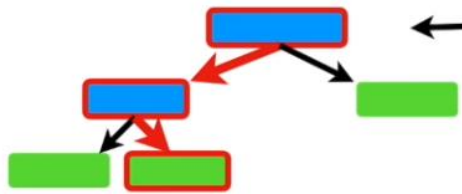


Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	No	210	No

Classification of the Out-Of-Bag sample	
Yes	No
1	3

Classification of the Out-Of-Bag sample	
Yes	No
4	0

Classification of the Out-Of-Bag sample	
Yes	No
3	1



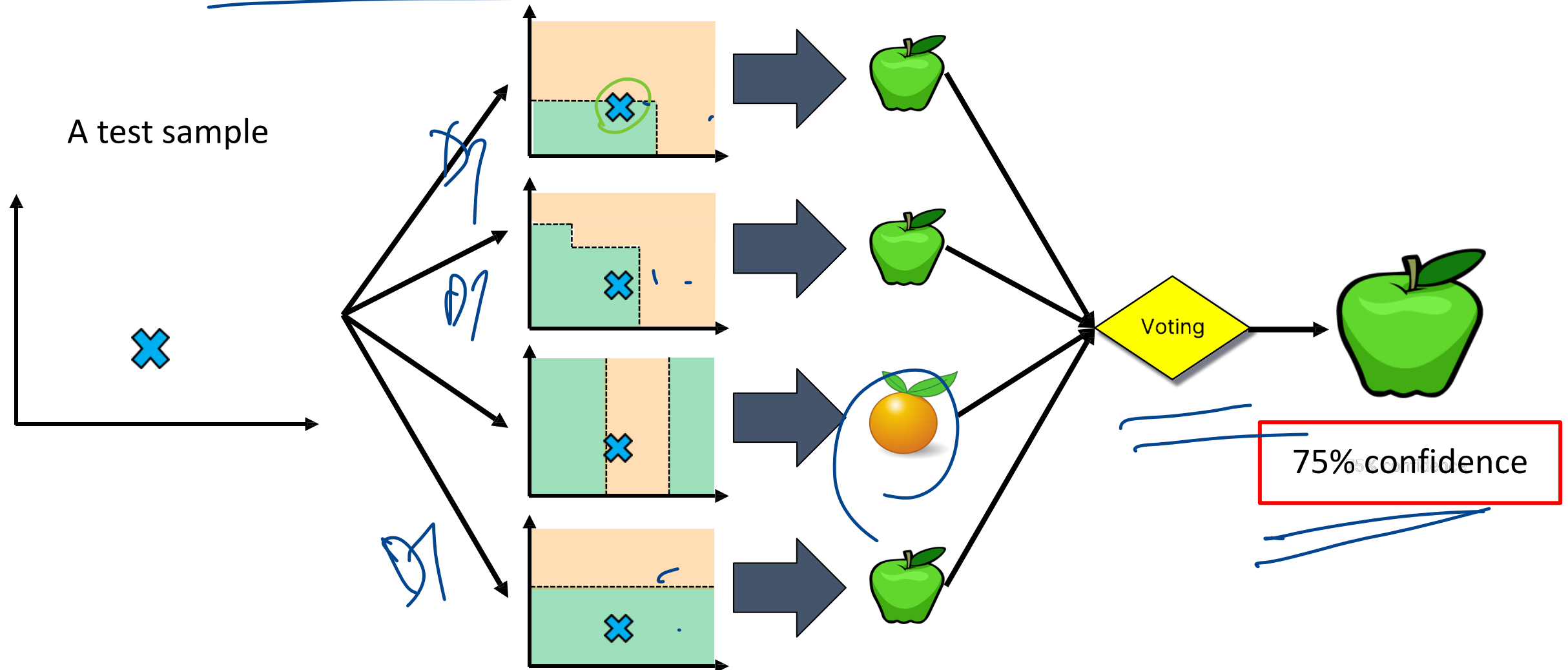
These trees correctly labeled the Out-of-Bag sample "No".

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No

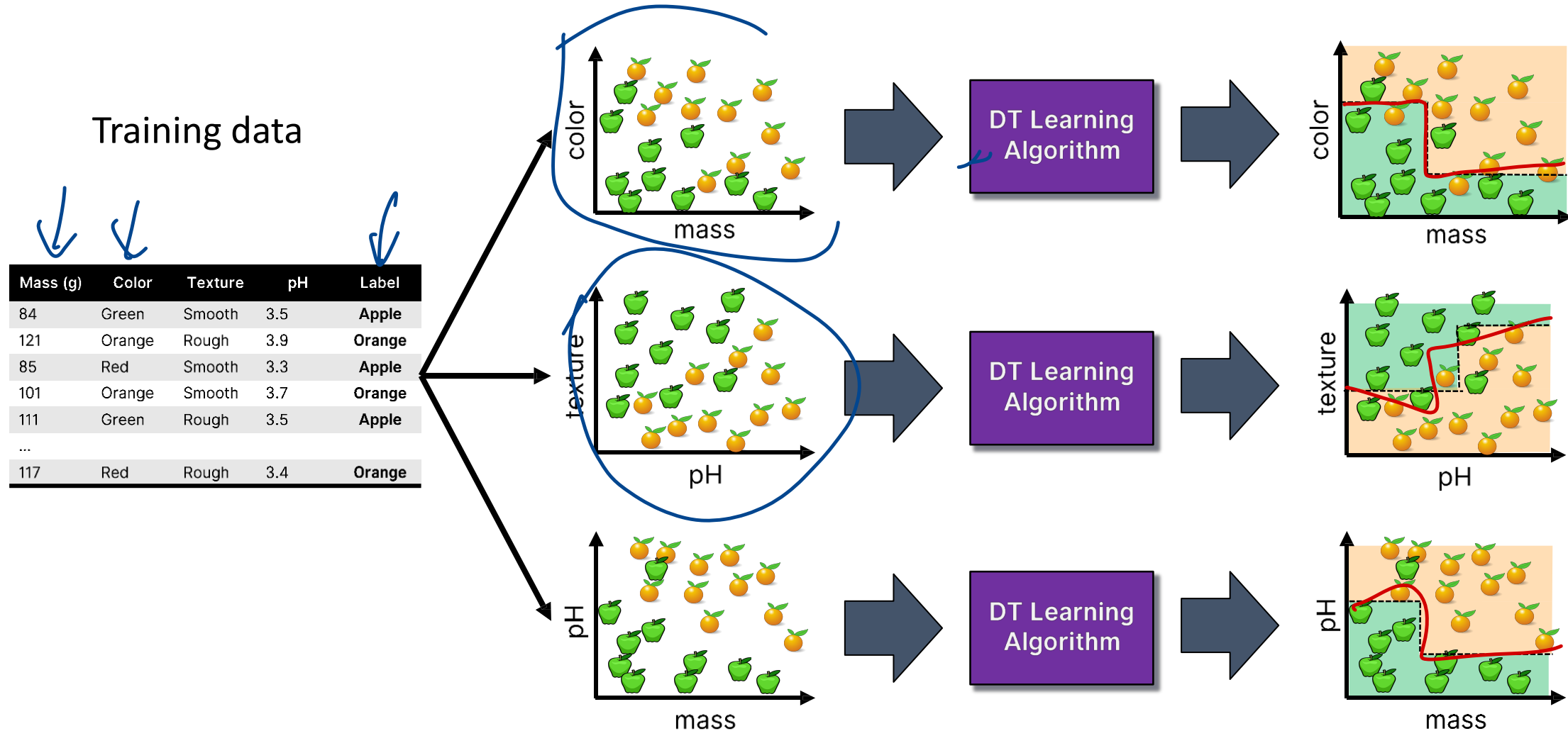
This Out-of-Bag sample was incorrectly labeled...

Bagging at Work - Inference

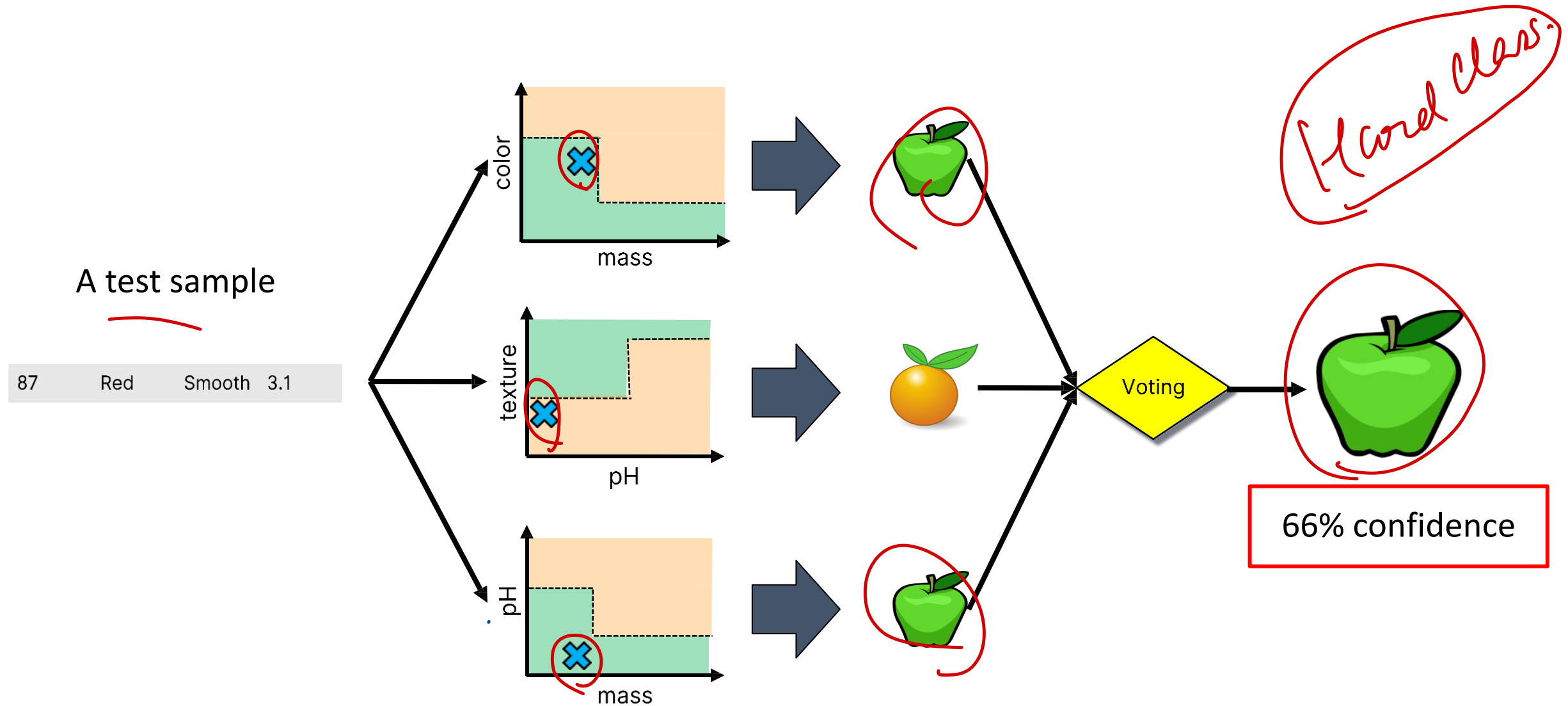
Output is the aggregate majority from all of the trees so created. Hence the name Bagging – Bootstrap Aggregating



Random Subspace Method at training time



Random Subspace Method at inference time



Random Forests

Mass (g)	Color	Texture	pH	Label
84	Green	Smooth	3.5	Apple
121	Orange	Rough	3.9	Orange
85	Red	Smooth	3.3	Apple
101	Orange	Smooth	3.7	Orange
111	Green	Rough	3.5	Apple
...				
117	Red	Rough	3.4	Orange



Bagging +
Random Subspace Method +
Decision Tree Learning Algorithm



Random Forests

Code:

```
from sklearn.ensemble import RandomForestClassifier  
rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,  
                                n_jobs=-1, random_state=42)  
rnd_clf.fit(X_train, y_train)  
y_pred_rf = rnd_clf.predict(X_test)
```

Feature Importance

Correlation

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
>>> rnd_clf.fit(iris.data, iris.target)
>>> for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
...     print(round(score, 2), name)
```

128

0.42 petal width

0.44 Petal length (cm)

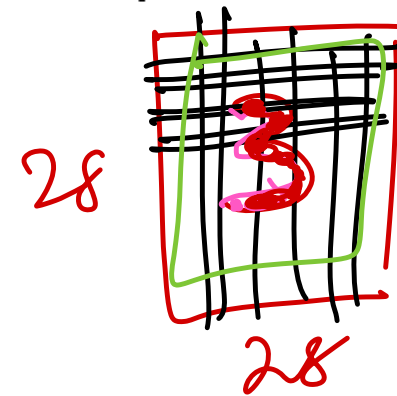
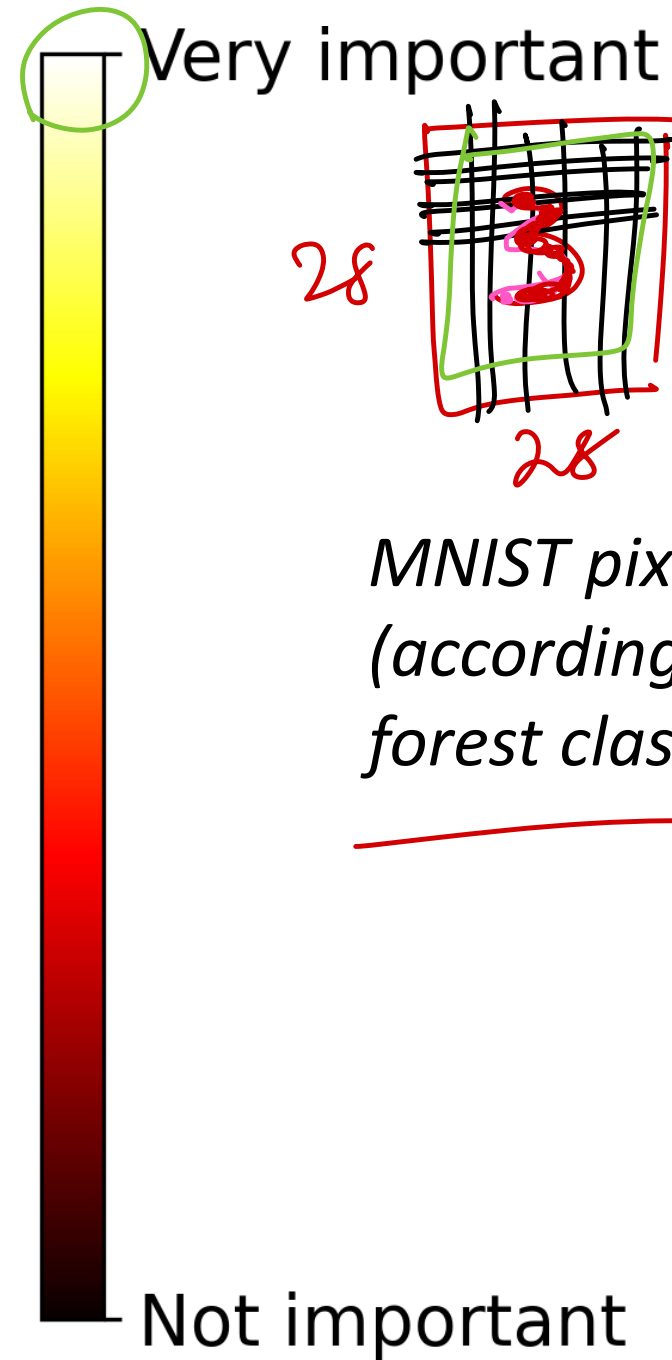
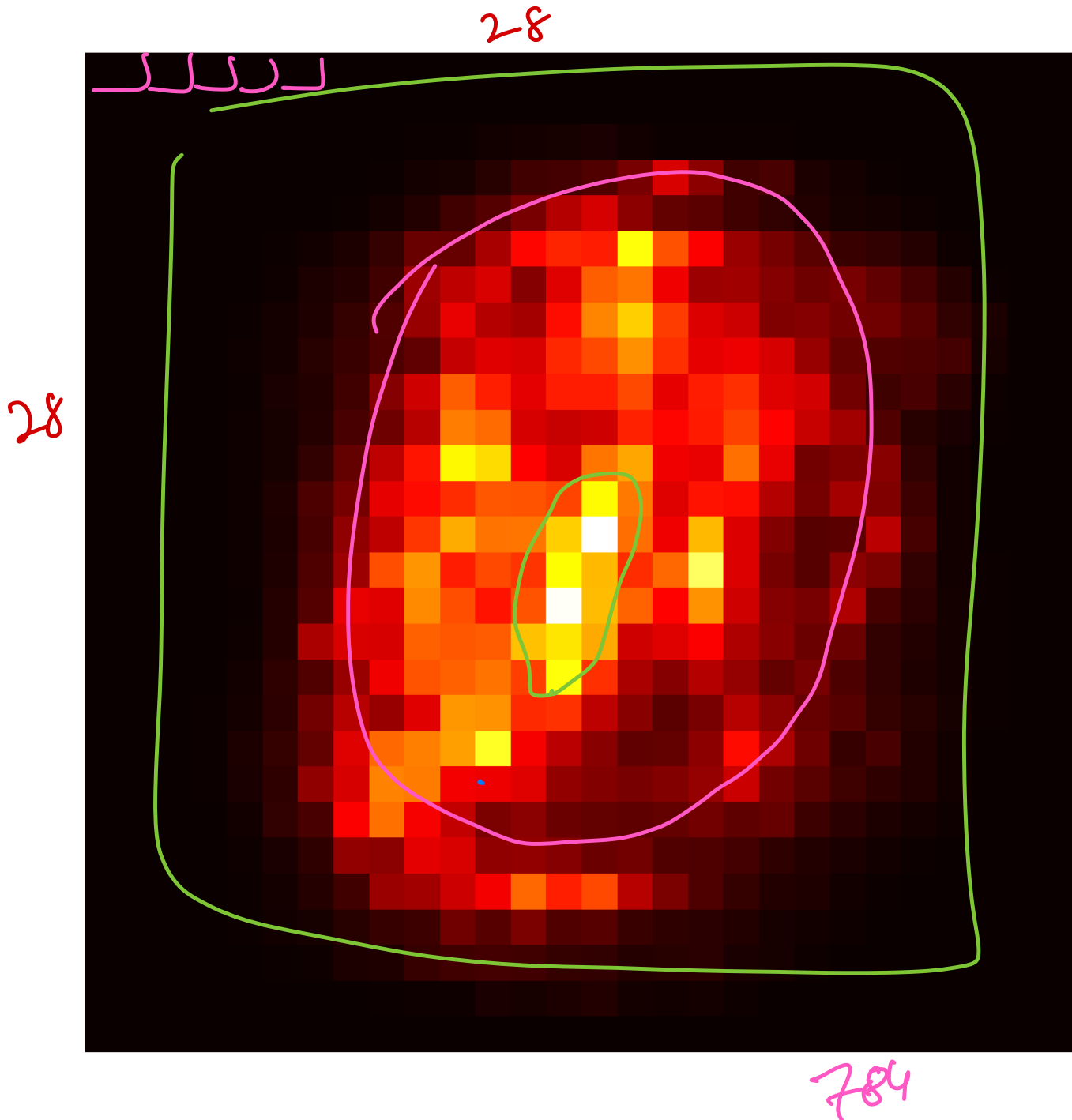
0.11 sepal length (cm)

0.02 sepal width (cm)

Scikit-Learn measures a feature's importance by looking at how much the tree nodes that use that feature reduce impurity on average, across all trees in the forest.

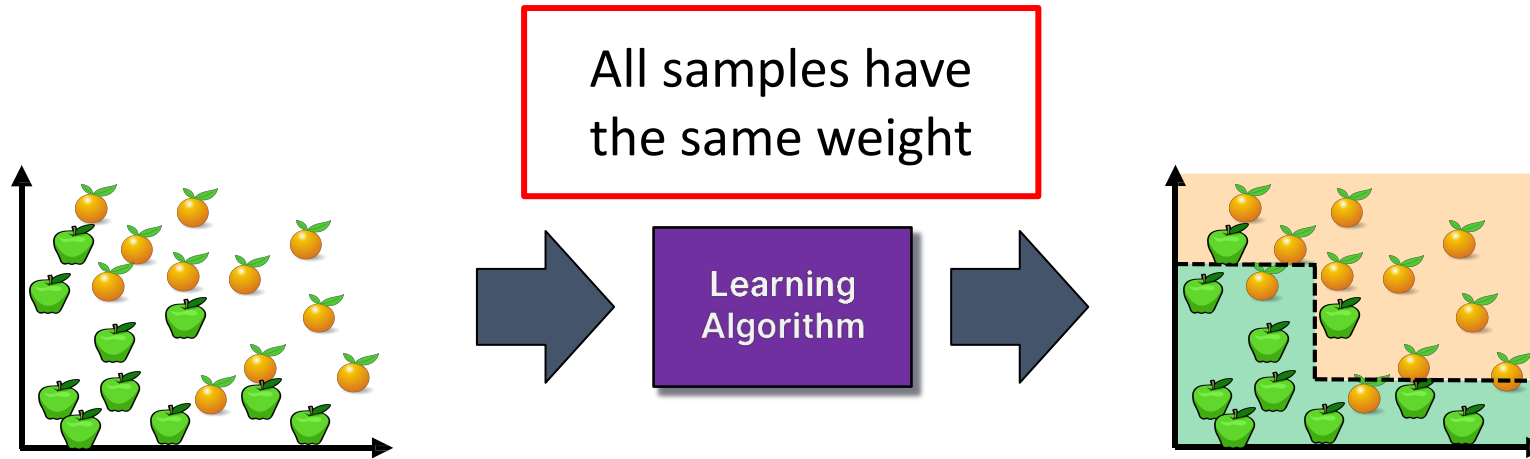
It seems that the most important features in the IRIS data set are the petal length (44%) and width (42%), while sepal length and width are rather unimportant in comparison (11% and 2%, respectively)

1%...



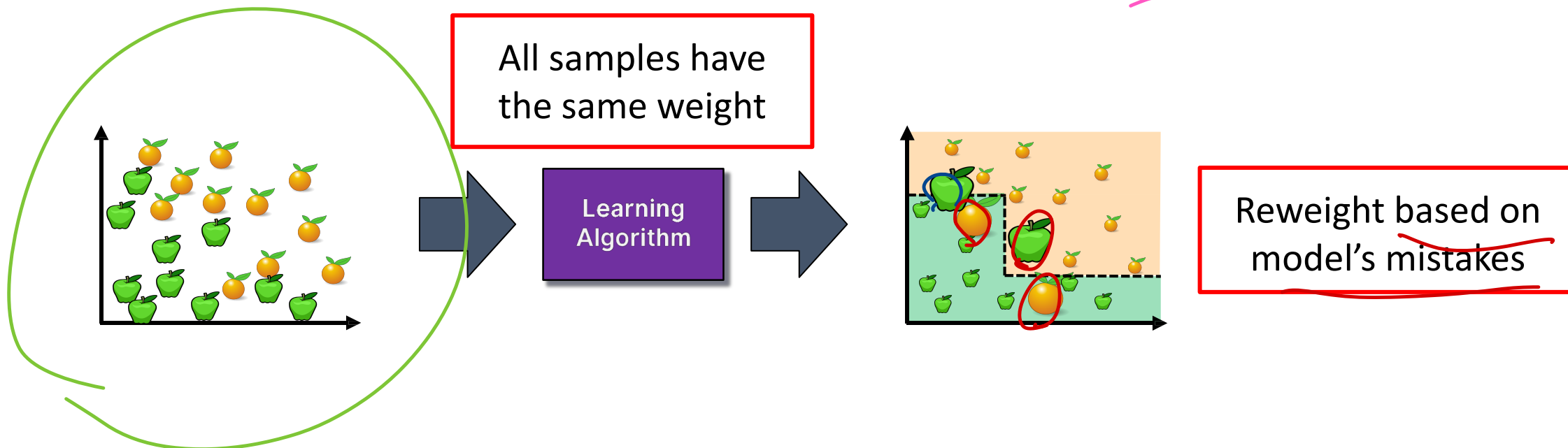
*MNIST pixel importance
(according to a random
forest classifier)*

Boosting

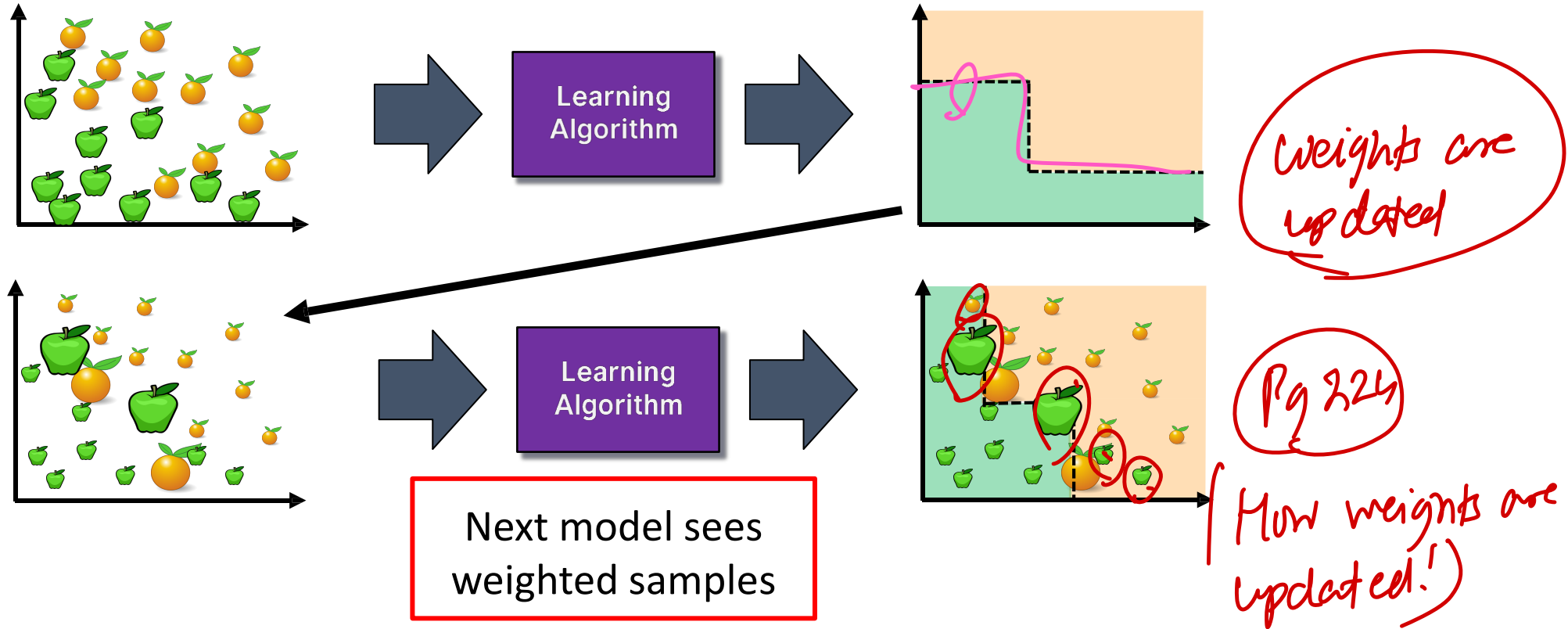


Boosting

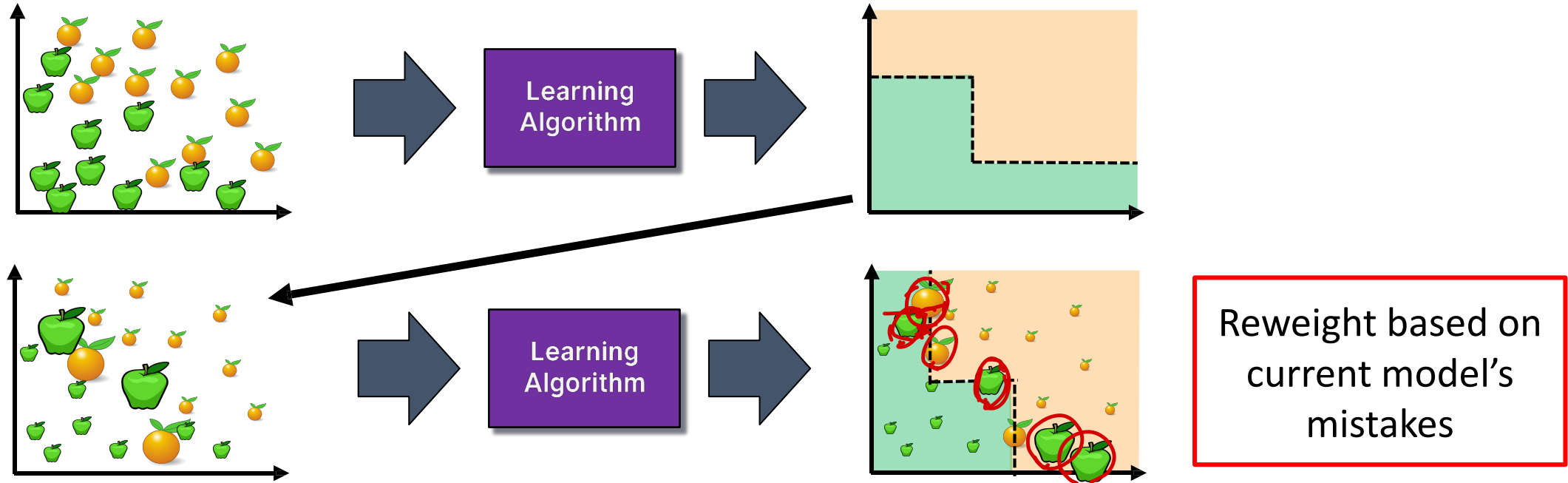
AdaBoost



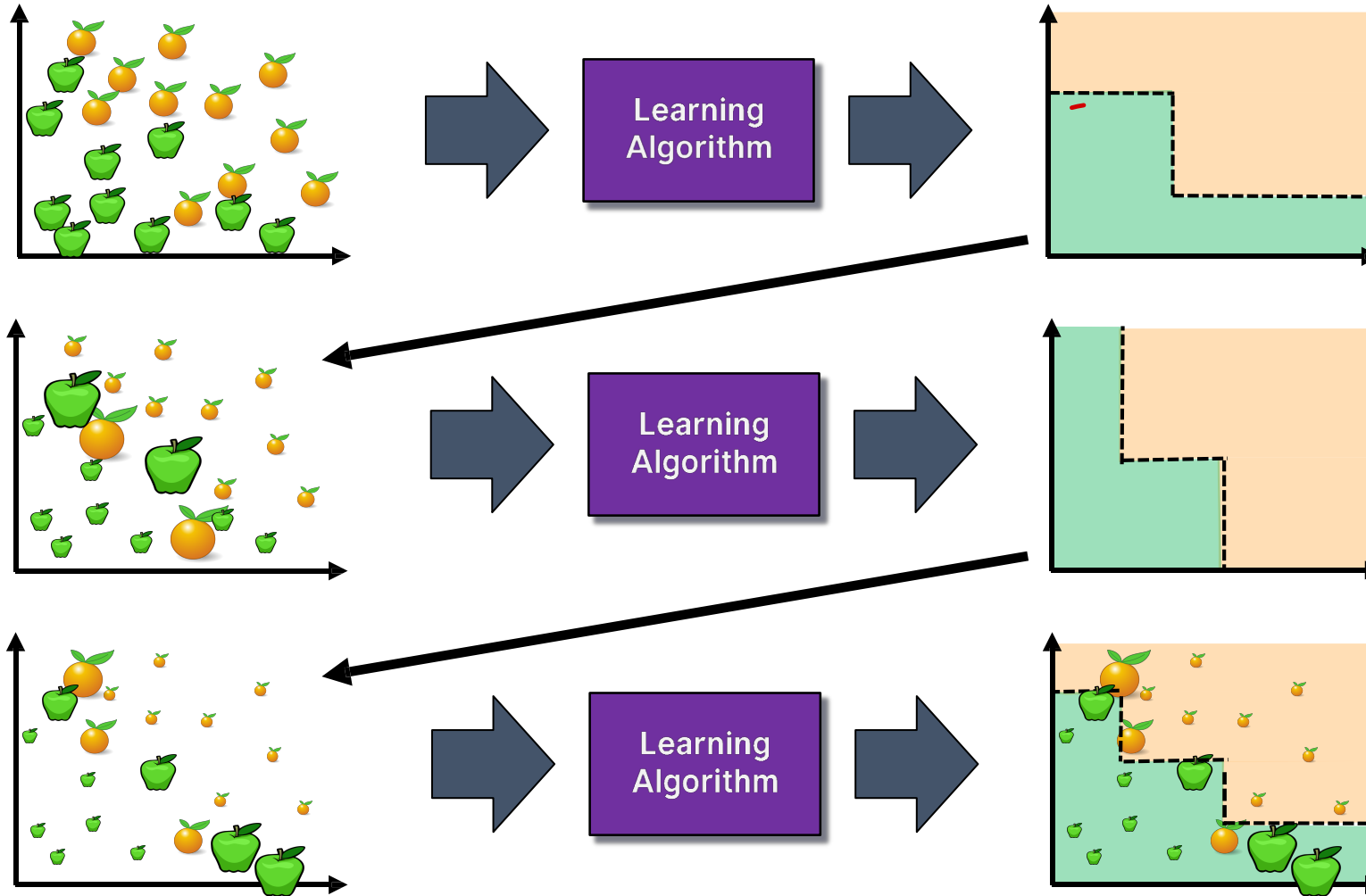
Ada - Boosting



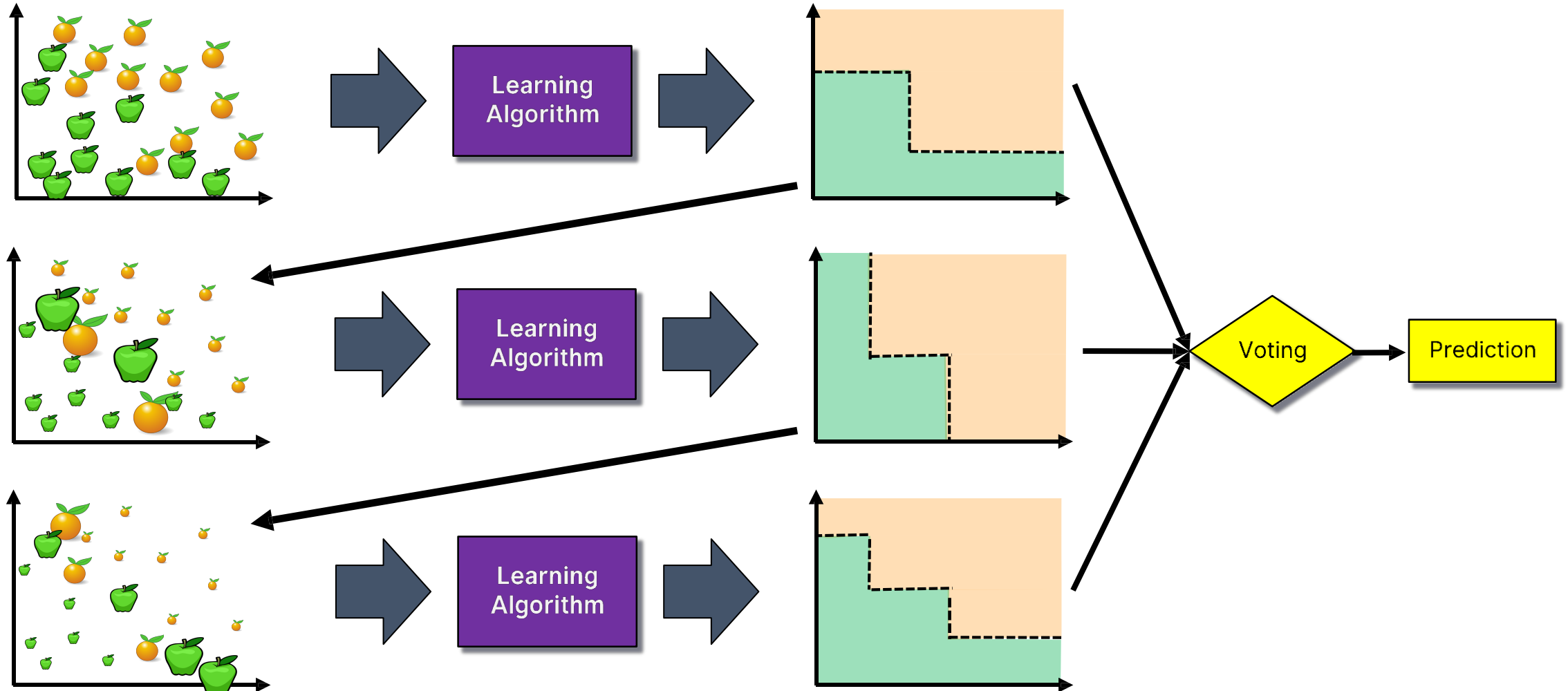
Boosting



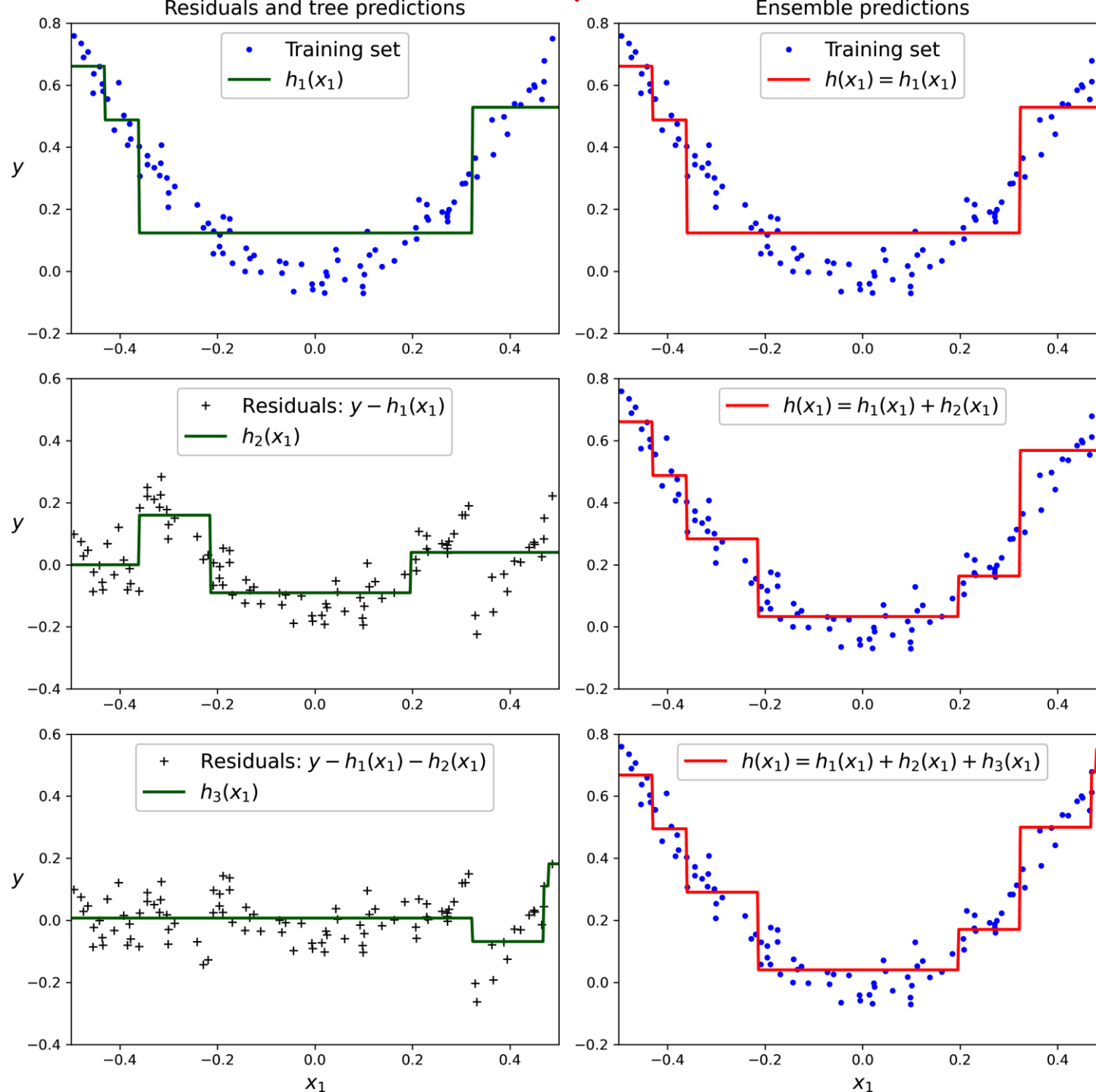
Boosting



Boosting



Gradient Boosting –
works on the residuals



Summary

- Ensemble Learning methods combine multiple learning algorithms to obtain performance improvements over its components
- Commonly-used ensemble methods:
 - Bagging (multiple models on random subsets of data samples)
 - Random Subspace Method (multiple models on random subsets of features)
 - Boosting (train models iteratively, while making the current model focus on the mistakes of the previous ones by increasing the weight of misclassified samples)
- **Random Forests** are an ensemble learning method that employ decision tree learning to build multiple trees through **bagging** and **random subspace method**.
 - They rectify the overfitting problem of decision trees!

History of Random Forests

- Introduction of the Random Subspace Method
 - “Random Decision Forests” [Ho, 1995] and “The Random Subspace Method for Constructing Decision Forests” [Ho, 1998]
- Combined the Random Subspace Method with Bagging. Introduce the term **Random Forest** (a trademark of Leo Breiman and Adele Cutler)
 - “Random Forests” [Breiman, 2001]

Decision Trees and Random Forest (Python)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

clf = DecisionTreeClassifier(criterion = "entropy", min_samples_leaf = 3)
# Lots of parameters: criterion = "gini" / "entropy";
#                       max_depth;
#                       min_impurity_split;

clf.fit(X, y) # It can only handle numerical attributes!
# Categorical attributes need to be encoded, see LabelEncoder and OneHotEncoder

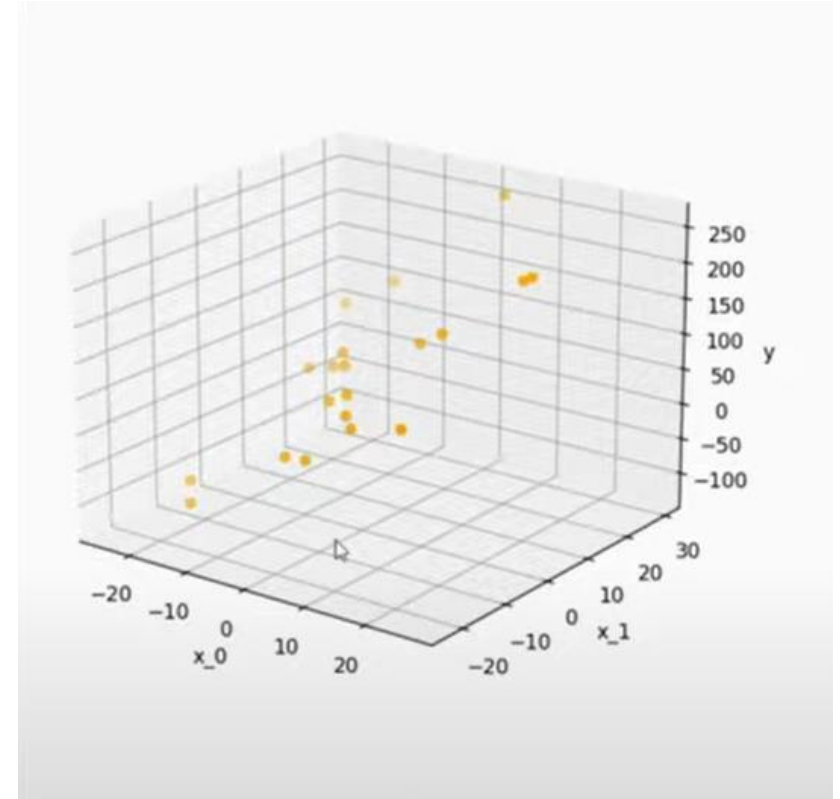
clf.predict([x]) # Predict class for x

clf.feature_importances_ # Importance of each feature
clf.tree_ # The underlying tree object

clf = RandomForestClassifier(n_estimators = 20) # Random Forest with 20 trees
```

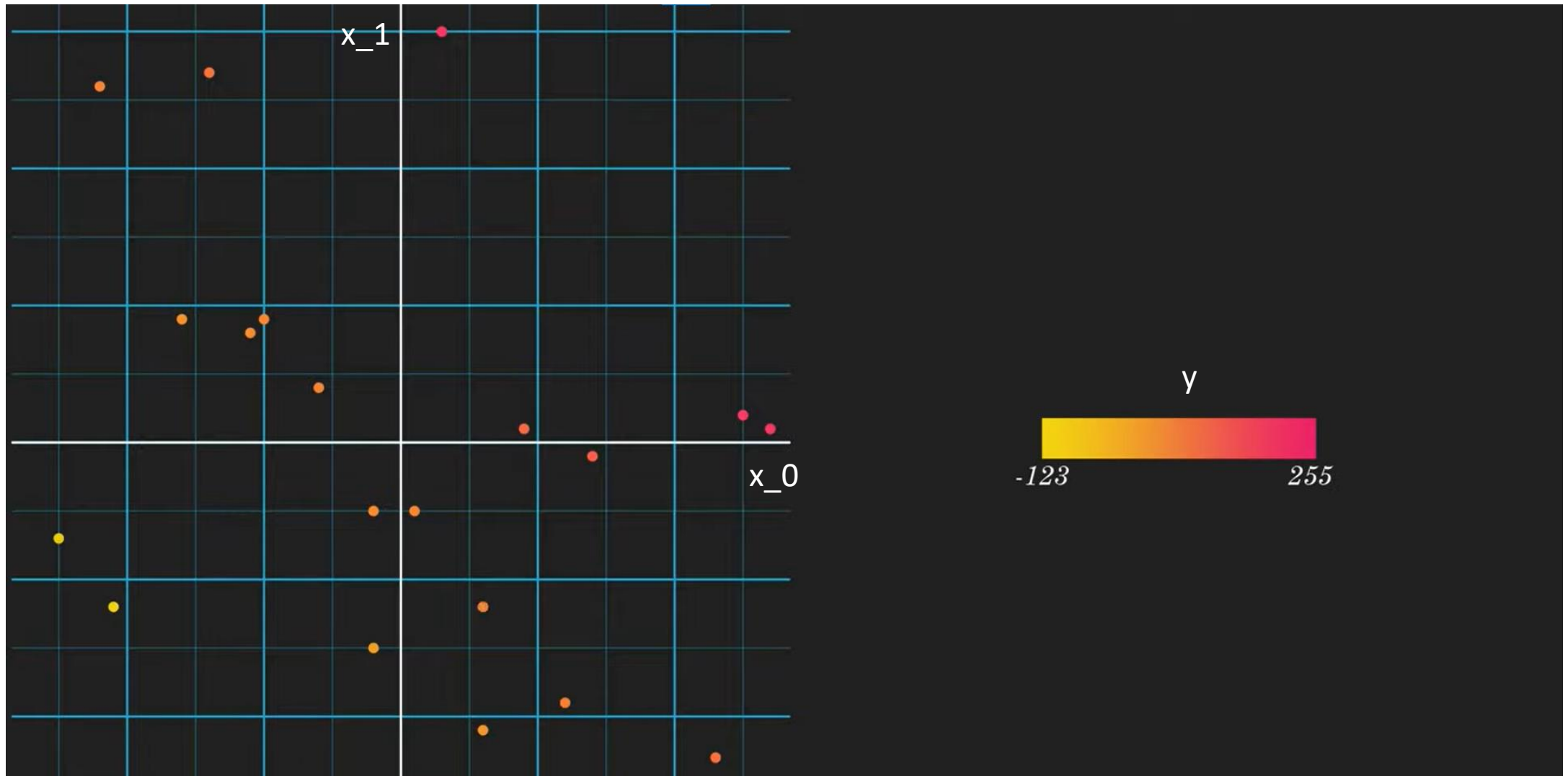
Regression Trees

- We used trees for classification tasks. But how do these fare with regression tasks?
- A tree based model (DT, RF etc.) that performs regression.
- What is the intuitive explanation behind regression within trees?
- How does one ensure a “best fitting” split of data within regression trees?
- What is an alternative to the GINI and Entropy index in classification trees?
- Examine with an example.
- Assume the data:
- Contains two features, x_0 and x_1 .
- Y is the target.



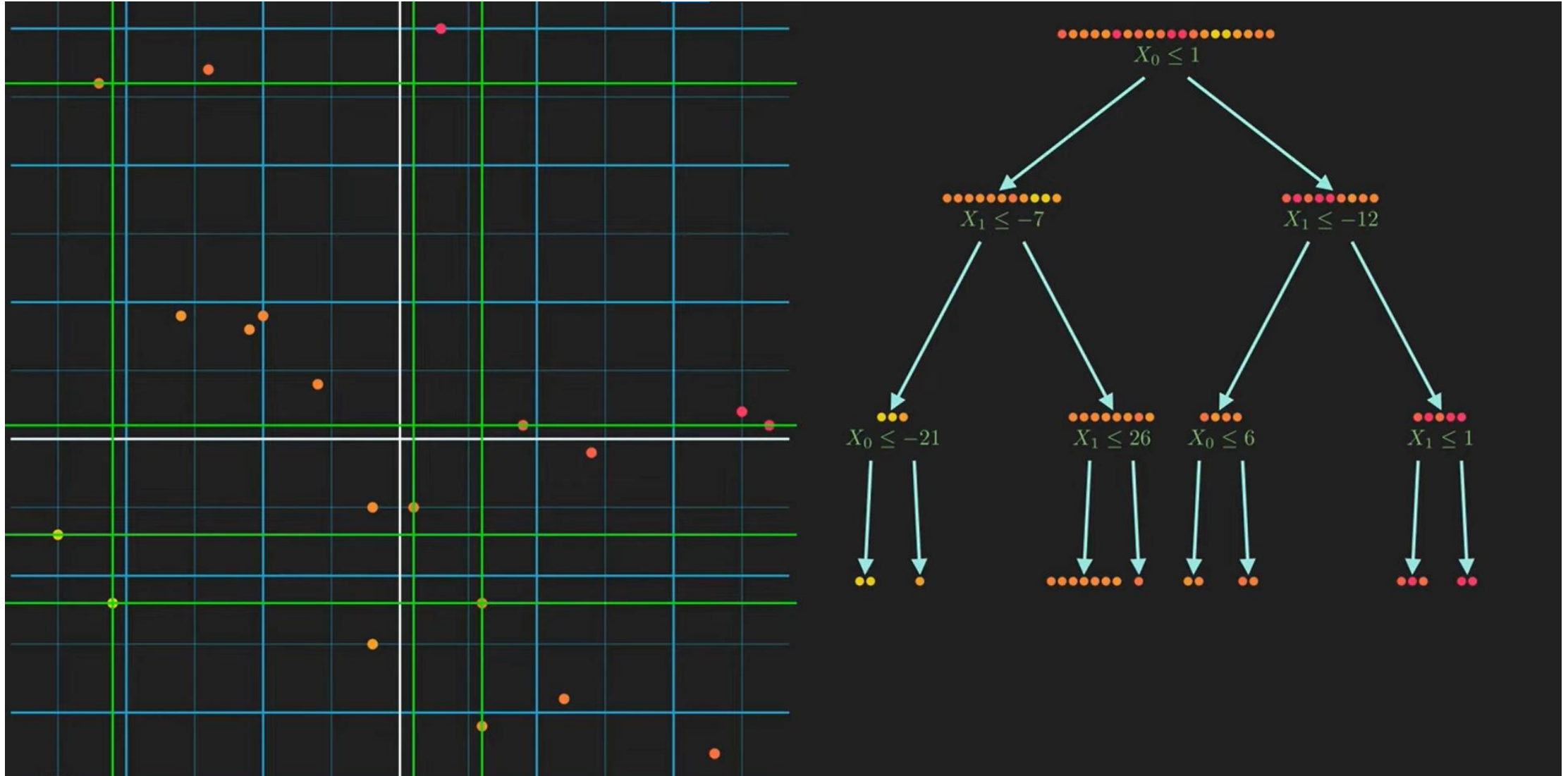
Regression Trees

- Representing in 2D with the colormap showing the y value,



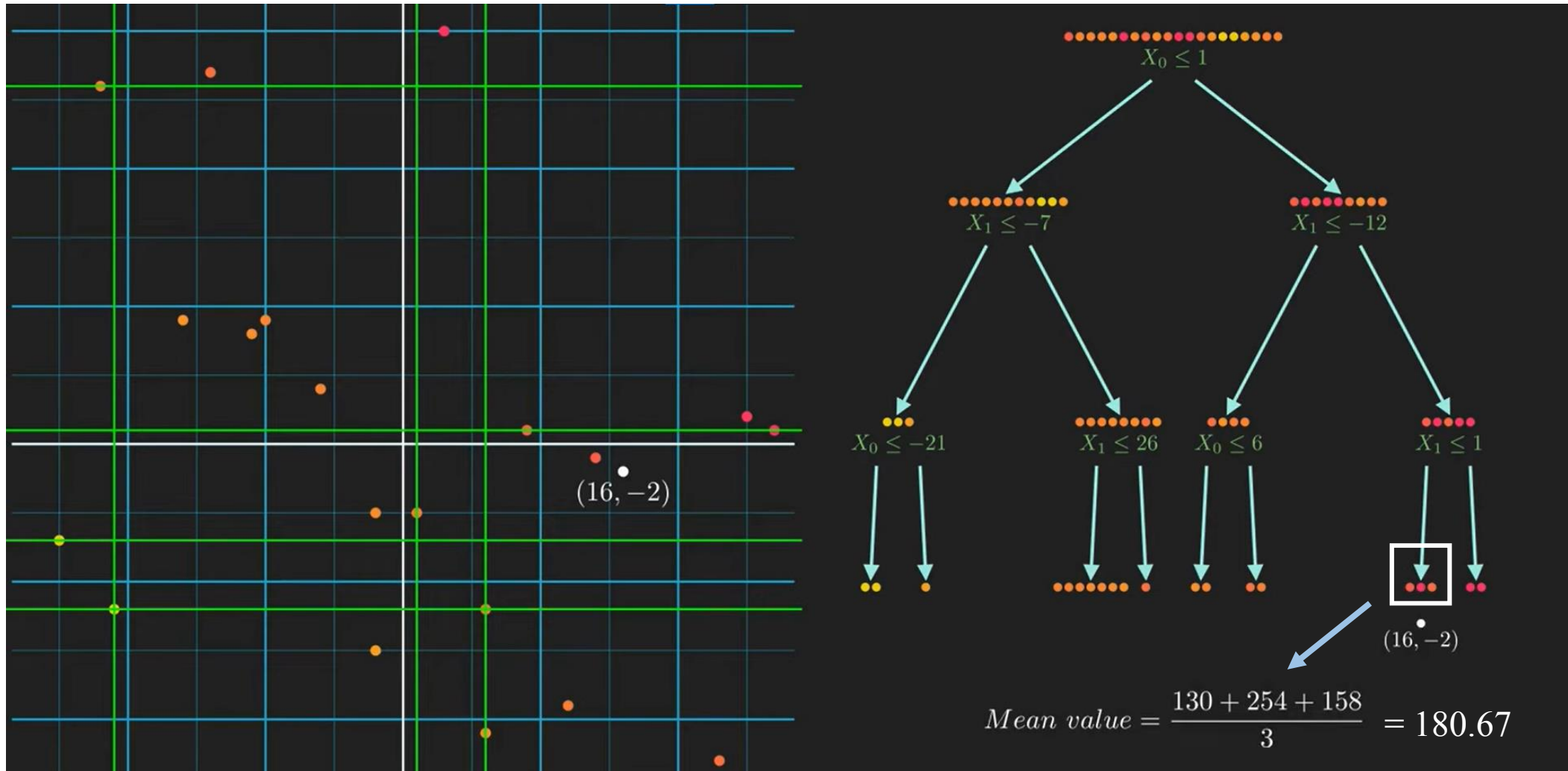
Regression Trees

- Sly to cDT, we recursively split until we obtain pure leaf nodes.



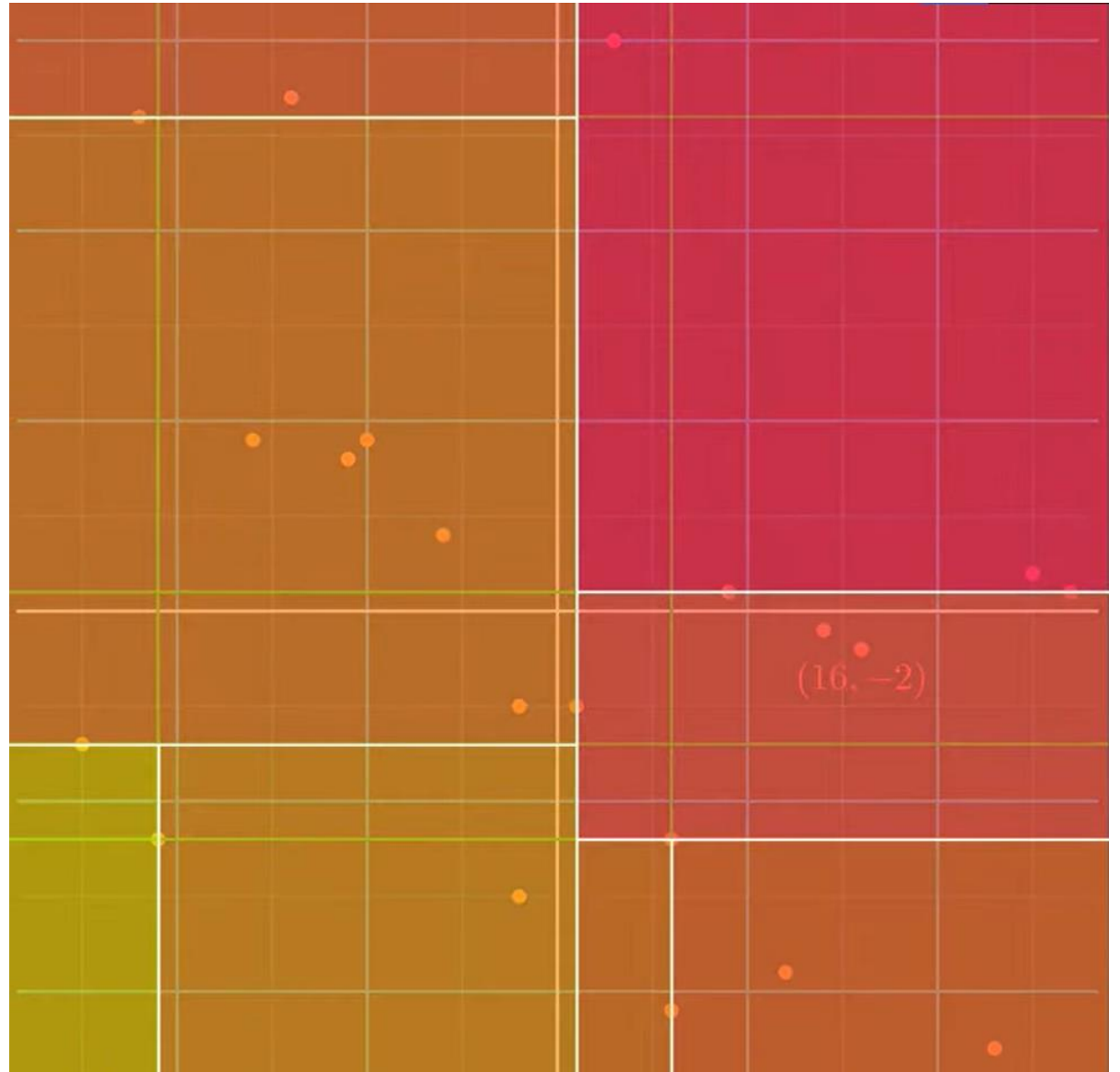
Regression Trees

- To predict, what is the prediction for 16, -2? Pass it through the tree. Find the average of values within the points of the leaf node.



Regression Trees

- It splits the feature space into divisions and the predictions are the average of the values within enclosing region.



Regression Trees

- But, multiple splits can be possible at each node. So, how to decide which split is the best? What is alternative to GINI/Entropy in cDT?
- This is done by performing a reduction in variation.
- The metric of variation over here is the MSE – mean squared error.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

The equation is annotated with blue boxes and labels: 'Mean' above the fraction, 'Error' above the difference term, 'Squared' above the square term, and a box around the entire summation term.

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

- Check MSE at each node for possible splits.
- Choose the split with least variance/least MSE.

Regression Trees

- Analysis of two splits is shown.
- Use split 2.
- Random forest regression is an extension of cRF similar to the method described in regression trees.
- The average of all random tree predictions is taken as the final prediction.

